



ITWM106- Mobile Application Development 3 (cross-platform)

HANDS-ON ACTIVITY 2

Name: _____

Score/Rating: _____

Schedule: _____

I. Simple Calculator using Flexbox, Custom Components, Props and State. 50 pts.

Please refer to uploaded PDF lecture in Canvas LMS (Lecture 3 to 5)

Develop a Simple Calculator App that can perform basic arithmetic operations.

Use flexbox, custom components, props and state.

Use React Native Flexbox properties to create the calculator layout.

Use TouchableOpacity for the calculator buttons.

Create at least one reusable custom component for the calculator interface. For example, you may create a component such as:

```
<NumButton num='2' onPress={() => { this.updateDisplay('2'); }} />
```

The component should be reused for multiple calculator buttons rather than creating every button separately.

Use props to pass information to your custom component.

Use React state to manage information that changes while the application is running.

Your calculator should include:

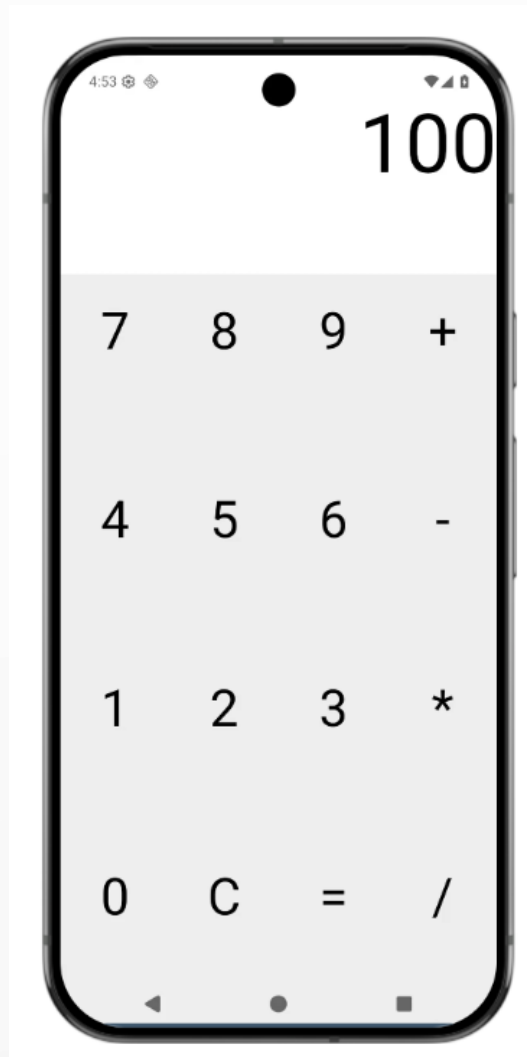
- A display area for showing the current input and/or result.
- Number buttons from 0 to 9.
- An addition (+) button.
- A subtraction (−) button.
- A multiplication (×) button.
- A division (÷) button.
- An equals (=) button.
- A clear (C or AC) button.

The calculator should be able to perform at least the following operations:

- Addition, Subtraction, Multiplication, Division



Sample Output:





Rubric

Criteria	Excellent 10 pts	Satisfactory 8 pts	Needs Improvement 5 pts	Points
Functionality	Correctly performs addition, subtraction, multiplication, and division; buttons and calculations work properly.	Most operations work but minor issues are present.	Several operations are missing or do not work properly.	10
Flexbox	10 pts Effectively uses Flexbox to create a well-organized and responsive calculator layout.	8 pts Uses Flexbox appropriately but has minor layout issues.	5 pts Minimal or incorrect use of Flexbox.	10
Custom Components	Creates a reusable and well-structured custom component that is effectively reused throughout the application.	Creates a custom component but reuse or structure could be improved.	Custom component is missing or not meaningfully used.	10
Props and State	Correctly passes and uses props to make the custom component reusable and dynamic. Correctly uses state to manage calculator input, operations, and/or results.	Props are used but implementation is limited or has minor issues. State is used but has minor implementation issues.	Props are missing or incorrectly implemented. State is missing or does not properly manage changing calculator data.	10
User Interface & Design	Calculator is clean, readable, organized, visually appealing, and easy to use.	Interface is functional and readable but has some design/layout issues.	Interface is difficult to use, poorly organized, or incomplete.	10